Apollo-RTOS: An AGC-Inspired Real-Time Operating System for Embedded Systems

M. Ahsan Al Mahir

Department of Computer Science
University of Oxford

Supervisor: Prof. Alex Rogers

A project report submitted for the degree of Master of Mathematics and Computer Science

Trinity 2025

1914 words (using `texcount`)

Abstract

Embedded systems have evolved dramatically, yet many of the challenges faced by early computing systems remain relevant today. The Apollo Guidance Computer (AGC), despite its limited hardware—a 2 MHz CPU, 4 KB of RAM, and 72 KB of ROM—introduced innovative scheduling and fault-tolerant mechanisms that ensured reliability in mission-critical environments. Inspired by the AGC, this project presents Apollo-RTOS, a real-time

operating system (RTOS) for the Microbit v1, designed to explore the applicability of these historical concepts in modern embedded computing.

Apollo-RTOS implements a hybrid scheduling algorithm, combining cooperative and preemptive multitasking to optimize resource utilization and responsiveness. Additionally, it features a fault-tolerant recovery system, ensuring process integrity even during failures. The system extends the AGC's original design by incorporating modern OS functionalities, including a file system leveraging flash memory and an external FRAM device, inter-process communication through signals, and a shell interface for system interaction. Implemented in modern C++, Apollo-RTOS utilizes RAII, templates, and type deduction to enhance efficiency and maintainability.

This project demonstrates how historical embedded computing techniques can be adapted to contemporary microcontrollers, offering insights into designing resilient, efficient, and resource-aware real-time operating systems for modern applications.

Contents

§1 Introduction

1.1 Background & Motivation

Computing capabilities have grown at an unprecedented rate over the last century. The **Apollo Guidance Computer** (AGC), which played a critical role in the Apollo program, operated with a modest *2 MHz CPU*, *4 KB of RAM*, and *72 KB of ROM*. To maximize its limited resources, the AGC's operating system introduced innovative solutions for *task scheduling*, *error handling*, and *resource-efficient computation*. One of its most groundbreaking features was a hybrid scheduling mechanism that combined cooperative and preemptive multiprogramming, ensuring that critical tasks could execute reliably even in adverse conditions. Additionally, the AGC incorporated a robust recovery mechanism, enabling fault tolerance in the face of system failures.

Today, a compact `micro:bit` device costing less than £20 possesses significantly more computing power than the AGC. Such microcontrollers are now ubiquitous, powering applications ranging from consumer electronics to mission-critical systems. Despite the vast improvements in computational power, many of the challenges that the AGC's operating system addressed remain highly relevant, particularly in constrained embedded environments where real-time performance, reliability, and fault tolerance are essential.

1.2 Project Objectives

The goal of this project is to study the design principles of the AGC's operating system and implement a robust real-time operating system (RTOS) for the `micro:bit` v1, named ***Apollo-RTOS***. This project aims to explore the *applicability of the AGC's novel scheduling techniques and fault-tolerant mechanisms* in modern embedded computing. Additionally, it extends these foundational ideas by incorporating *contemporary operating system concepts* to enhance usability, efficiency, and resilience.

Apollo-RTOS integrates:

- **Hybrid scheduling** that balances cooperative and preemptive multitasking.
- **A fault-tolerant system** capable of handling hardware failures and ensuring process recovery.
- Separate **file systems** implemented on flash memory and an external FRAM device for persistent storage.
- **Inter-process communication** via signals for dynamic event handling.
- **A shell interface** for interacting with the system and managing processes.
- **A modern C++ implementation** leveraging RAII, templates, and type deduction for robustness and efficiency.

By bridging historical embedded computing principles with modern software engineering techniques, this project provides insights into designing efficient, fault-tolerant operating

systems for resource-constrained environments.

§2 The Apollo Guidance Computer

The Apollo Guidance Computer (AGC) was an early digital computer developed in the 1960s to help navigate and control Apollo spacecraft on their missions to the Moon. It provided real-time guidance and control, making it essential for the success of the Apollo program.

The AGC ran at a speed of 2.048 MHz, which translated to about 40,000 instructions per second. The AGC had a 16-bit `word` length, 15 bits for data, and one parity bit. It had 2 kilowords or 4KB of erasable magnetic-core memory (RAM) along with 36 kilowords or 72KB of fixed core rope memory (ROM) for storing software and mission data. Astronauts interacted with the AGC using the Display and Keyboard (DSKY), a 21-digit display with a 19-button keyboard that allowed them to input commands and receive information. This interface was critical for making manual adjustments and monitoring the system during missions.

Due to these constraints, the operating system, called the Executive, had to efficiently manage resources. It ensured that important tasks were prioritized over less critical ones and that key functions could continue even if the system experienced a reset due to an error.

The Executive was written in AGC assembly, where each instruction was 16 bits long. Because the instruction set was small, the system also included a sophisticated interpreter that acted as a virtual machine, providing more advanced pseudo-instructions beyond the basic AGC instructions.

The details of the AGC Executive explained below is taken from O'Brien [1].

2.1 The Executive

The central part of the AGC software was the Executive, that ensured that processes and tasks got a fair share of the system resources like cpu time and dynamic memory. To seemlessly perform the real-time control and navigation tasks with extreme reliability and efficient, the Executive implemented a **priority-based cooperative scheduling** algorithm. The algorithm is described below:

- The highest running process capitalizes the CPU
- Lower priority processes will wait until the highest priority process either
  - lowers its priority, or
  - goes to `sleep` while waiting for some event to happen
- While the current highest running process is running, if a new process with a higher priority gets scheduled,
  - the new process waits until the previous process lowers its priority or goes to `sleep`,
  - then the new process starts running

Core sets Information about the running processes were stored in a table called Core Sets, which is basically what we call process descriptors nowadays. Each core set had a structure similar to (but not exactly):

```
struct CORE_SET {
    word priority;
    word program_counter;
    word bbank_reg; // area in unswitched memory where the code is
    word vac_ptr;   // pointer to extra memory

    word mpac[7];   // multi-purpose accumulators
    word mpac_mode;
};
```

Each running process must occupy one of the core sets. There were

- 6 available core sets in the Command module, and
- 7 available core sets in the Lunar module.

In either table, the core set 0 contained the currently running process. So a context switch basically was equivalent to swapping out a core set with the core set 0.

Addressable memory The 7 `mpac` words had two purposes. In the normal mode, they gave 7 words of free memory to the programmer to use as they saw fit. In the interpreter mode, they worked just like accumulator registers, used to store values of different calculations by the interpreter. If a process needed more memory, it could request for 42 extra words from the **Vector Accumulator** area.

Starting new process To start a new process, an empty core set must be found first, followed by finding a VAC area if requested. After succesfully populating the core set, the Executive must decide if the new process has a higher priority.

A special register `NEWJOB` is used just for this purpose.

- If the value of `NEWJOB` is 0, then the currently running process has the highest priority,
- if the value is non-zero, say `d`, then the process represented by the core set at `d` is the highest priority process.

So when a new process is created, its priority is compared with the priority of the current process, and the `NEWJOB` register is updated accordingly.

Switching processes A process context switch only occurs when the current process gives up execution, given that the current process continues to probe `NEWJOB` every 20ms for a higher priority process. If the `NEWJOB` register is not checked for over 640ms, it is assumed that the current process has hung up, and a hardware restart must be issued to deal with the hangup.

To switch the process, the Executive just needs to store the current context in the core set at 0, and swap it with the core set at the value stored in `NEWJOB`. Since the program counter is read from the core set directly, this automatically resumes the switched in process from it's

last instruction.

If the current process is still the highest running process, and it needs to go to sleep, or it decreases it's priority, a scan of the core sets table is made to find the next highest priority process, followed by a switch.

Sleep and Wakeup Processes can voluntarily give up execution as it waits for an external event to occur by going to sleep. The `JOBSLEEP` instruction puts the currently running process to sleep. An asleep process is represented by a negative priority. So all `JOBSLEEP` needs to do is negate the currently running processes' priority, and find the next highest priority job.

On the other hand, once the awaited even occurs, another process can wake up a sleeping process using the `JOBWAKE` instruction. This instruction negates the negative priority of the asleep process, which marks it as runnable. It then checks with the `NEWJOB` register to decide the highest priority process.

A process can call `DELAYJOB` instruction to go to sleep for a certain amount of time. To achieve this, the instruction schedules an "alarm" task using the **waitlist**, which is described below.

2.1.1 The Waitlist

The core processes in the Executive are all co-operative, and even the highest priority process won't preempt the currently running process until it gives up the execution. Waitlist provides preemptive scheduling capabilities to the AGC. Tasks that need to be done within a particular deadline can be scheduled using the waitlist. Every 10ms, a timer interrupt is generated, which runs any waitlist task that needs to be run.

In essence, the waitlist is a ***sorted linked list*** of tasks in increasing order of their deadline. When a new waitlist task t is scheduled with a particular deadline, it is inserted into the linked list at the correct position so that

- any task t_0 with a deadline less than t appears before t in the list, and
- any task t_1 with a deadline greater than t appears after.

Every 10ms, the timer interrupt checks if the deadline of the first task in the list is about to expire, and runs that task if so, and then removes the task from the list.

The implementation of waitlist in the Apollo-RTOS is modeled completely after the implementation in the AGC, so the implemenation details will be discussed later.

2.2 Interpreter

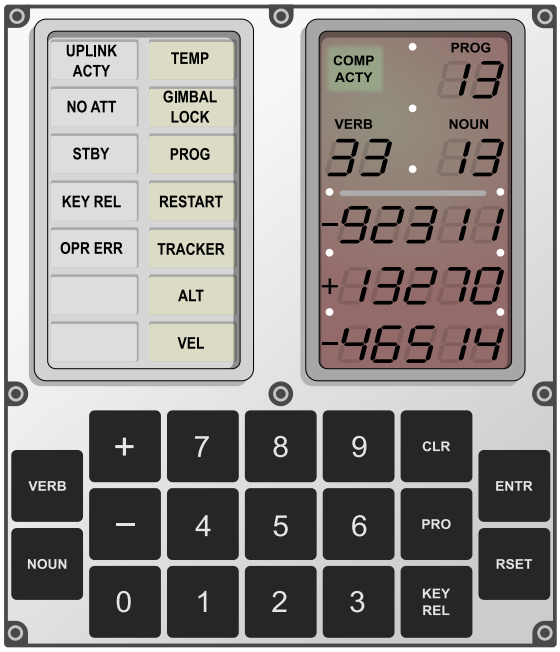

Figure 0.1: DSKY interface diagram by Oona Räisänen

2.3 Design philosophy of AGC

Unlike most modern general-purpose computers, the AGC operated on a tightly defined world of hardware and software, where every part of the system could be "trusted", in the sense that every part would behave as expected. As a result, it was deemed unnecessary to truly isolate the hardware and the OS from the application code. Because of this, the operating system could make many assumptions limiting the scope of services, limiting the complexity in design. This resulted in a small but efficient operating system.

This theme of mutual trust within the operating system will be prevelant throughout the project. Since the scope of a microcontroller is quite minimal, and resources are limited, it makes sense to not make any kernel/user space isolation. All usages of system resources is already anticipated during development. However, to minimize possibility of programming errors, it's necessary to enclose system resources under system and library calls to minimize human programming errors.

Figure 0.2: Margaret Hamilton standing next to listings of the software she and her MIT team produced for the Apollo Project.

2.3.1 Design consequences

There are a few consequences of this design of the scheduler:

Since all the processes use the same address space, and inter process communication is done through shared memory

Letting the currently running process decide when to give up the execution context fixes several nasty race conditions usually prevelant in preemptive schedulers. For example,

- In multi-threaded pro

§3 micro:bit v1 Architecture and Specifications

(i) Hardware overview (nRF51822 microcontroller, memory, I/O)

(ii) Constraints and capabilities

(iii) Comparison with AGC hardware

  (i) Processing power

  (ii) Memory availability

  (iii) I/O capabilities

  (iv) Power requirements

(iv) Development environment and toolchain

§4 Implementation Architecture and Design

(i) Overall system architecture

(ii) Design principles and approach

(iii) Adaptations for micro:bit constraints

(iv) High-level organization and module relationships

§5 Implementation Details

5.1 Core Executive System

(i) Task scheduling and prioritization
(ii) Interrupt handling
(iii) Time management
(iv) Original AGC features preserved vs. modified

5.2 Memory Management

(i) Memory organization and allocation
(ii) FRAM extension implementation
(iii) Comparison with AGC's memory management

5.3 I/O Subsystem

(i) Interface with micro:bit peripherals
(ii) Mapping AGC I/O concepts to micro:bit
(iii) Novel I/O approaches

5.4 Key Algorithms and Procedures

(i) Implementation of critical AGC algorithms
(ii) Performance optimizations
(iii) Code examples of significant components

§6 Evaluation and Testing

(i) Testing methodology on micro:bit
(ii) Performance metrics
(iii) Reliability considerations
(iv) Comparison with original AGC capabilities
(v) Limitations and trade-offs

§7 Future Work

(i) Potential improvements and extensions

(ii) Unresolved challenges

(iii) Research or application possibilities

§8 Conclusion

(i) Summary of achievements

(ii) Reflections on the AGC's enduring design principles

(iii) Broader implications for OS design

Reference

[1] Frank O'Brien. ***The Apollo Guidance Computer: Architecture and Operation***. 01 2010. ISBN 978-1-4419-0876-6. doi: 10.1007/978-1-4419-0877-3.

§A Appendices

(i) Code snippets of key algorithms

(ii) Detailed specifications

(iii) Test results